

Dynamic and Temporal GNNs

Piotr Lipiński

Introduction

Dynamic Graph Learning

- Most real-world graphs evolve over time.
- Nodes, edges, and attributes may appear, disappear, or change.
- Node or edge feature may change with time.

Examples:

- Social networks
- Financial transaction networks
- Recommender systems
- Communication networks
- Cybersecurity event graphs

Static Nature of GNNs

- Classical GNNs (GCNs, GATs, Graph Transformers) assume a fixed graph $G = (V, E)$ and static node/edge features
- No explicit temporal modeling:
 - no evolving nodes
 - no changing edges
 - static features
- Real-world graphs are inherently dynamic

Consequences of Static Modeling

- Loss of temporal information
- Information leakage in future prediction tasks
- Inconsistent node representations over time
- Limitations in:
 - traffic prediction
 - link prediction in social networks
 - fraud detection
 - recommender systems

Problem Formulation

Dynamic Graph Neural Networks (DGNNs) explicitly model graph evolution.

Dynamic graph:

$$G_t = (V_t, E_t, X_t)$$

where graph topology and node attributes evolve over time.

Two major settings:

Discrete-Time

$$G_1, G_2, \dots, G_T$$

Sequence of graph snapshots.

Continuous-Time

$$(u, v, t, m)$$

Temporal interaction occurring at time (t).

Learning Tasks

Node-level:

- Node classification
- Node state forecasting
- Anomaly detection

Edge-level:

- Link prediction
- Temporal link prediction
- Interaction recommendation

Graph-level:

- Event forecasting
- Dynamic graph classification

Taxonomy of Dynamic GNNs

Four dominant paradigms:

1. Recurrent Dynamic GNNs
2. Parameter-Evolving GNNs
3. Attention-Based Temporal GNNs
4. Memory-Based Temporal GNNs

Continuous-time models often combine:

- Memory
- Attention
- Temporal point processes

Types of Dynamic Graphs

- Snapshot-based: $G_1, G_2, \dots, G_T$
- Event-based: individual interactions (u, v, t)
- Continuous-time: graph function $G(t)$

Dynamic GNNs

Idea of Dynamic GNNs

- Evolving node embeddings over time
- Modeling graph sequences
- Snapshot-based formulation:

$$h_v^{(t)} = f(h_v^{(t-1)}, G_t)$$

Representative Models

- EvolveGCN:
 - evolution of GCN parameters via RNN
- Limitations:
 - coarse temporal granularity
 - snapshot discretization issues

Temporal GNNs

Temporal Message Passing

- Time-dependent messages:

$$m_{u \rightarrow v}(t) = f(h_u, h_v, \Delta t)$$

- Key component: time encoding

Temporal GNN Architectures

- TGAT (Temporal Graph Attention Network)
- JODIE (Joint Dynamic User-Item Embedding)
- DyRep (Dynamic Representation Learning over Dynamic Graphs)
- TGN (Temporal Graph Networks)
- and others

Time Encoding Strategies

- Sinusoidal encoding
- Learnable time embeddings
- Fourier features
- Relative time bias in attention

TGAT

TGAT: Temporal Graph Attention Networks

Observation:

- Snapshot discretization introduces information loss.
- Events occur at irregular times.

Goal:

Learn node representations directly in continuous time.

Functional Time Encoding

Time is embedded using harmonic functions.

$$\phi(t) = [\cos(\omega_1 t), \dots, \cos(\omega_k t)]$$

Properties:

- Continuous representation
- Multi-scale temporal patterns
- Similar to Transformer positional encoding

Temporal Attention Mechanism

Attention score:

$$e_{ij} = a(h_i, h_j, \phi(\Delta t))$$

Normalized coefficient:

$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_k \exp(e_{ik})}$$

Aggregation:

$$h'_i = \sum_j \alpha_{ij} h_j$$

Temporal distance directly affects message importance.

TGAT Training Objective

Positive interaction:

$$(u, v, t)$$

Negative samples:

$$(u, v^-, t)$$

Binary objective:

$$\mathcal{L} = \log(1 - \sigma(z_u^\top z_{v^-}))$$

Used for temporal link prediction.

JODIE

Designed for bipartite interaction streams (User-Movie, User-Product, User-Post, etc.).

Joint evolution of users and items in the embedding space (continuous trajectories).

JODIE (Joint Dynamic User-Item Embedding)

Two coupled recurrent networks:

$$RNN_{user}$$

$$RNN_{item}$$

Interaction event:

$$(u, i, t)$$

updates both embeddings simultaneously.

Projection Operator

Unique contribution:

Future embedding prediction.

$$\hat{h}_u(t + \Delta t) = \rho(h_u(t), \Delta t)$$

Allows prediction before the next interaction occurs.

Important for recommendation systems.

DyRep

Dynamic Representation Learning.

Models two processes:

1. Communication events
2. Association events

These evolve at different temporal scales.

Temporal Point Processes

Intensity function:

$$\lambda(t)$$

Probability density:

$$p(t) = \lambda(t) \exp \left(- \int_0^t \lambda(\tau) d\tau \right)$$

Interpretation:

- Predict who interacts
- Predict when interaction occurs

Event:

$$(u, v, t)$$

causes:

- embedding updates
- neighborhood influence propagation
- structural evolution

Representations evolve continuously rather than per snapshot.

Temporal Graph Networks

Temporal Graph Networks (TGN)

Unified framework for temporal graph learning.

Components:

1. Memory
2. Message Function
3. Aggregator
4. Memory Updater
5. Embedding Module

Node Memory

For each node:

$$s_i(t)$$

stores historical information.

Message:

$$m_i(t)$$

Memory update:

$$s_i(t) = GRU(s_i(t^-), m_i(t))$$

Memory acts as a compressed event history.

Message Construction

For event:

$$(u, v, t, e_{uv})$$

message:

$$m = f(s_u, s_v, e_{uv}, t)$$

Typically includes:

- source memory
- destination memory
- edge features
- temporal encoding

Embedding Computation

Node embedding:

$$z_i(t) = \textit{Embed}(s_i(t), N(i))$$

Possible embedding modules:

- Graph Attention
- GraphSAGE
- Identity Mapping

Separates memory management from representation learning.

For every event:

1. Read memories.
2. Compute message.
3. Aggregate messages.
4. Update memory.
5. Compute embeddings.
6. Optimize prediction loss.

Complexity scales with the number of observed events.

Graph Transformers for Dynamic Graphs

Why Transformers for Graphs?

- Limitations of message passing:
 - locality constraint
 - over-smoothing
- Self-attention enables global reasoning

Graph Transformer Idea

- Nodes treated as tokens
- Attention replaces neighborhood aggregation
- Graph structure encoded as bias or mask

- Graphormer

Challenge: Temporal Dynamics

- Standard Graph Transformers are static
- Cannot model:
 - evolving edges
 - event sequences

- Time-aware attention:

$$A_{ij} = f(h_i, h_j, \Delta t)$$

- Event-based tokenization: (u, v, t)
- Memory + attention hybrid architectures

Advantages and Limitations

Advantages:

- global dependencies
- strong long-range modeling

Limitations:

- quadratic complexity $O(N^2)$
- scalability issues
- high memory cost

Summary

When to Use What?

- Dynamic GNNs → snapshot-based evolution
- Temporal GNNs → event-driven fine-grained time
- Graph Transformers → global reasoning long-range dependencies